KFS — Data Dictionary

Every table in the schema, explained in plain English: what it's *for*, and what each column actually *means* day-to-day — not just its SQL type.

Domain Glossary

Everyday terms used throughout KFS, defined once here so the per-table sections below don't have to keep re-explaining them.

Term	Plain-English meaning
Entry	A single piece of captured knowledge — a note, a recipe, anything you've saved. The core "thing" in the system.
Knowledge Base	A top-level container for Entries — think "a notebook" or "a workspace." Everyone gets one automatically (their **Inbox**); they can create more to keep areas of their life/work separate.
Inbox	The one Knowledge Base every user gets automatically, marked as their *default*. New Entries land here until organized elsewhere.
Node	A folder-like item inside a Knowledge Base, used to organize Entries into a tree. Nodes can nest inside other Nodes.
Filing	The act of placing an Entry under a specific Node. An Entry can be filed under more than one Node at once (it's a many-to-many placement, not a single "location").
Tag	A short label attached to an Entry for cross-cutting categorization (independent of the Node tree). Scoped to one Knowledge Base — the tag "urgent" in one KB is unrelated to "urgent" in another.
Relationship	A typed connection between two Entries — e.g. "this references that," "this precedes that." Independent of both the Node tree and Tags; this is the knowledge *graph*, not the *hierarchy*.
Content Type	What "kind" of Entry this is (a plain Note, a Recipe, etc.), which determines what extra fields it has.
Source	How an Entry came to exist — typed by hand, imported in bulk, or AI-suggested.
Status	Where an Entry sits in its lifecycle: freshly captured (Inbox), organized (Active), or put away (Archived).
Attachment	A file or external link attached to an Entry.
Comment	A note left on an Entry by the owner or a collaborator — separate from the Entry's own content.
Owner	The user who created and controls a Knowledge Base. Always has full access; never needs an explicit access grant.
Contributor / Viewer	The two levels of access a Knowledge Base owner can grant to someone else. Viewers can look; Contributors can also add Comments and Attachments — but not create Entries, Nodes, Tags, or Relationships, which stay owner-only.

Term	Plain-English meaning
Archived	A soft-delete state — the record is hidden from normal use but not physically deleted, so it can be recovered or audited later.
System user	A built-in, non-human user account (`username = 'system'`) used to attribute data that KFS itself creates (like the built-in Content Types), rather than falsely crediting a real person.
Version	A hidden counter used for optimistic locking — protects against two people overwriting each other's edits at the same time without either seeing an error.

Reference / Vocabulary Tables

These are small, mostly-fixed lookup tables — the "pick one of these options" lists used throughout the schema. Their rows are seeded when the database is created and rarely change afterward.

`system_role`

Purpose: The list of platform-wide authority levels a user can hold — exactly one at a time.

Column	Everyday meaning
`id`	Internal identifier.
`code`	The short machine-readable name (`ADMIN`, `USER`, `SYSTEM`) other tables actually reference.
`name`	The human-friendly display name ("Administrator").
`description`	A sentence explaining what the role means.
`display_order`	What order to show these in on a screen (dropdowns, etc.).
`active`	Whether this role can still be assigned to anyone (a way to retire a role without deleting history).

`entry_status`

Purpose: The lifecycle stages an Entry moves through.

Column	Everyday meaning
`code`	`INBOX`, `ACTIVE`, or `ARCHIVED` — see Glossary.
`name`, `description`, `display_order`, `active`	Same pattern as `system_role` above.

`source`

Purpose: How an Entry originally came into existence.

Column	Everyday meaning
`code`	`MANUAL` (typed by hand), `IMPORT` (bulk-loaded), or `AI_SUGGESTED`.
`name`, `description`, `display_order`, `active`	Same pattern.

`relationship_type`

Purpose: The vocabulary of ways two Entries can be connected in the knowledge graph.

Column	Everyday meaning
`code`	e.g. `RELATED_TO`, `REFERENCES`, `REQUIRES`, `PRECEDES`, `FOLLOW_UP`, `SUPERSEDES`, `PART_OF`, `CAUSED_BY`.
`symmetric`	Whether the relationship reads the same in both directions. `RELATED_TO` is symmetric ("A relates to B" = "B relates to A"); `REQUIRES` is not ("A requires B" doesn't mean "B requires A").
`name`, `description`, `display_order`, `active`	Same pattern.

`access_level`

Purpose: The two levels of access a Knowledge Base owner can grant to someone else (the owner's own access is implicit and never stored as a row here).

Column	Everyday meaning
`code`	`VIEWER` (read-only) or `CONTRIBUTOR` (can also add Comments/Attachments).
`name`, `description`, `display_order`, `active`	Same pattern.

`attachment_type`

Purpose: Whether an Attachment points at an actual stored file, or just an external link.

Column	Everyday meaning
`code`	`FILE` (something KFS is storing) or `LINK` (a URL pointing elsewhere, e.g. a YouTube video).
`name`, `description`, `display_order`, `active`	Same pattern.

`user`

Purpose: Every person (or system account) who can own or interact with data in KFS. This is the root every other table's "who did this?" columns point back to — which is exactly why it doesn't have its own `created_by`/`modified_by`: there's no other user to credit a user's own creation to.

Column	Everyday meaning
`id`	Internal identifier, referenced everywhere as `owner_id`, `created_by`, etc.
`username`	The person's unique handle/login name.
`email`	Their unique email address.
`system_role_id`	Which `system_role` they hold (Admin, User, or System).
`created_at`	When the account was created. Plain timestamp — no `created_by`, for the reason above.

`knowledge_base`

Purpose: A top-level container for Entries — a "notebook" belonging to one owner. Every user has exactly one marked as their default (Inbox); they can create as many additional

ones as they like.

Column	Everyday meaning
`id`	Internal identifier.
`name`	The Knowledge Base's display name (e.g. "Personal," "Work Projects").
`description`	Optional longer explanation of what this KB is for.
`owner_id`	Which user owns it — the one person with full, always-on authority over it.
`is_default`	`TRUE` for exactly one Knowledge Base per owner — their automatic Inbox.
`created_at` / `created_by`	When and by whom this KB was created.
`last_modified` / `modified_by`	When and by whom it was last edited, if ever.
`archived_at` / `archived_by`	When and by whom it was archived (soft-deleted), if ever.
`version`	Optimistic-locking counter (see Glossary).

Content Type System

`content_type`

Purpose: Defines what "kind" of Entry something is, and therefore what extra fields it carries. Some kinds are built into KFS itself (like Recipe); others can be defined by a user on the fly.

Column	Everyday meaning
`code`	Short machine name, e.g. `NOTE`, `RECIPE`.
`name`, `description`	Display name and explanation.
`system_defined`	`TRUE` = a built-in type shipped with KFS (backed by its own dedicated table, like `recipe_detail`). `FALSE` = a type a user invented themselves at runtime.
`owner_id`	Who invented this type — always empty for built-in types, always filled in for user-invented ones.
`display_order`, `active`	Display ordering and whether it's still usable.
`created_at` / `created_by`	Standard creation audit.
`archived_at` / `archived_by`	Standard archive audit.

`content_type_attribute`

Purpose: For a user-invented Content Type only, this is where its custom fields are defined — e.g. "this user-made 'Book Review' type needs a Rating field, a number, and it's required." Built-in types like Recipe don't use this table at all; their fields are real columns on their own extension table instead.

Column	Everyday meaning
`content_type_id`	Which user-invented Content Type this field belongs to.
`attribute_name`	The field's name, e.g. "Rating."
`data_type`	What kind of value it holds: `TEXT`, `NUMBER`, `DATE`, or `BOOLEAN`.
`display_order`	What order to show these fields in on a form.
`required`	Whether this field must be filled in.
`created_at` / `created_by` / `last_modified` / `modified_by`	Standard audit columns.

`entry`

Purpose: The core table — one row per piece of captured knowledge. Everything else in KFS ultimately exists to organize, tag, connect, comment on, or attach files to rows in this table.

Column	Everyday meaning
`id`	Internal identifier.
`knowledge_base_id`	Which Knowledge Base this Entry lives in.
`title`	Optional short title.
`content`	The actual body/text of the Entry.
`status_id`	Where it sits in its lifecycle — Inbox, Active, or Archived.
`source_id`	How it came to exist — typed by hand, imported, or AI-suggested. Required on every Entry.
`content_type_id`	What kind of Entry it is (Note, Recipe, etc.). Required on every Entry.
`custom_attributes`	A flexible JSON bucket holding the values for a *user-invented* Content Type's fields (see `content_type_attribute`). Empty/unused for built-in types like Recipe, which store their fields in a real extension table instead.
`created_at` / `created_by`	When and by whom the Entry was captured.
`last_modified` / `modified_by`	When and by whom it was last edited.
`version`	Optimistic-locking counter.

`recipe_detail`

Purpose: The extra fields specific to an Entry whose Content Type is Recipe. Exists as a separate table (rather than extra columns on `entry` itself) so that Entry stays generic and only Recipes carry recipe-specific weight. One row per Recipe-typed Entry, sharing the same ID.

Column	Everyday meaning
`entry_id`	Which Entry this recipe detail belongs to (also its primary key — a strict 1-to-1 pairing).
`prep_time_minutes`	How long prep takes.
`cook_time_minutes`	How long cooking takes.
`servings`	How many servings the recipe makes.
`difficulty`	A free-text difficulty label.

Organization (the Tree)

`node`

Purpose: A folder-like item used to organize Entries into a hierarchy within a Knowledge Base. Nodes can nest inside other Nodes to build a tree.

Column	Everyday meaning
`knowledge_base_id`	Which Knowledge Base this Node belongs to.
`name`	The folder's display name.
`parent_node_id`	The Node one level up in the tree — empty for a top-level Node.
`display_order`	Where this Node sits among its siblings.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set.

`entry_node`

Purpose: Records that a specific Entry has been filed under a specific Node. This is the *placement* record — an Entry only counts as "organized" (out of the Inbox conceptually) once it has at least one row here, and it can have several if it's filed under more than one Node.

Column	Everyday meaning
`entry_id` / `node_id`	The Entry and the Node it's filed under (together, the primary key — one filing per pair).
`display_order`	Where this Entry sits among the other Entries filed under the same Node.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set — lets a single filing be tracked/archived independently of the Entry or Node it connects.

`relationship`

Purpose: A typed, directed connection between two Entries — the knowledge *graph*, separate from the Node *tree*. Answers "what is this connected to?" rather than "where does this live?"

Column	Everyday meaning
`source_entry_id`	The Entry the connection starts from.
`target_entry_id`	The Entry it points to.
`relationship_type_id`	What kind of connection this is (References, Requires, Precedes, etc.).
`notes`	Optional free-text explanation of why these two are connected.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set.

(Two rules enforced by the database itself: an Entry can't be related to itself, and the same exact source/target/type combination can't be recorded twice.)

Collaboration Layer

`knowledge_base_access`

Purpose: Grants someone other than the owner permission to view or contribute to a Knowledge Base. The owner's own access is automatic and is never recorded here.

Column	Everyday meaning
`knowledge_base_id`	Which Knowledge Base access is being granted to.
`user_id`	Who's being granted access.
`access_level_id`	What level — Viewer or Contributor.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set (e.g. tracking when access was revoked).

`comment`

Purpose: A note left on an Entry — by the owner, or by anyone granted Contributor access to that Entry's Knowledge Base. Kept separate from the Entry's own `content` so the Entry's core text and the discussion around it don't get mixed together.

Column	Everyday meaning
`entry_id`	Which Entry this comment is attached to.
`content`	The comment text itself.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set.

`file`

Purpose: A registry of physical files KFS has stored on disk, one row per unique file *content* — not per attachment. The same file can be attached to several different Entries (even across different Knowledge Bases) without KFS storing duplicate copies.

Column	Everyday meaning
`file_name`	The original filename.
`file_path`	Where it's stored on disk, relative to the configured storage root.
`mime_type`	What kind of file it is (e.g. `image/png`).
`file_size`	Size in bytes.
`content_hash`	A fingerprint of the file's actual bytes — used to detect "this exact file is already stored" and avoid duplicates. Must be unique.
`created_at` / `created_by` / `archived_at` / `archived_by`	Audit columns (no `last_modified`/`modified_by` — a stored file's bytes aren't edited in place; a changed file is a new File row).

`attachment`

Purpose: Connects an Entry to either a stored File or an external link — the user-facing "attachment," as opposed to `file`'s behind-the-scenes physical registry.

Column	Everyday meaning
`entry_id`	Which Entry this is attached to.
`attachment_type_id`	Whether this is a stored File or an external Link.
`file_id`	If it's a File-type attachment, which `file` row it points to. Empty for Links.
`external_url`	If it's a Link-type attachment, the URL. Empty for Files.
`display_name`	What to actually show the user (must be unique per Entry — no two attachments on the same Entry can share a display name).
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by` / `version`	Standard full audit set.

(The database enforces that exactly one of `file_id`/`external_url` is set — never both,

never neither.)

Tagging

`tag`

Purpose: A short label that can be attached to Entries for cross-cutting categorization, independent of the Node tree. Scoped to one Knowledge Base, so the same word can mean something different in two different KBs, and stays owner-authored only (unlike Comments/Attachments, which Contributors can also add).

Column	Everyday meaning
`knowledge_base_id`	Which Knowledge Base this Tag belongs to.
`name`	The tag's text, e.g. "urgent." Unique within its Knowledge Base — the same name can't be created twice in the same KB.
`created_at` / `created_by` / `last_modified` / `modified_by` / `archived_at` / `archived_by`	Full audit set, minus `version` (no optimistic-locking counter on this one).

`entry_tag`

Purpose: Records that a specific Tag has been applied to a specific Entry. Deliberately minimal compared to `entry_node` — since tagging is a simple "on or off" action authored only by the owner, there's no need for a `display_order` or `version`.

Column	Everyday meaning
`entry_id` / `tag_id`	The Entry and the Tag it's been given (together, the primary key — one tagging per pair).
`added_at`	When this tag was applied.
`created_by`	Who applied it.